

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Методы защиты информации»

ОТЧЁТ
к лабораторной работе №1
на тему
**«СИММЕТРИЧНАЯ КРИПТОГРАФИЯ. СТАНДАРТ ШИФРОВАНИЯ
ГОСТ 28147-89»**

БГУИР 6-05-0612-02 098

Выполнил студент группы 353502
ПЕТРОЧЕНКО Михаил Максимович

(дата, подпись студента)

Проверил ассистент кафедры информатики
ГЕРЧИК Артём Вадимович

(дата, подпись преподавателя)

Минск 2026

СОДЕРЖАНИЕ

1	Цель работы	3
2	Ход работы	4
2.1	Режимы ввода и вывода	4
2.2	Режим простой замены	5
2.3	Режим гаммирования	6
2.4	Режим гаммирования с обратной связью	6
2.5	Генерация имитовставки	7
2.6	Обработка файлов	7
	Заключение	9
	Приложение А (обязательное) Листинг программного кода	10

1 ЦЕЛЬ РАБОТЫ

Целью данной лабораторной работы является изучение принципов построения симметричных блочных шифров на примере отечественного стандарта шифрования ГОСТ 28147-89. Задачи включают изучение структуры сети Фейстеля, лежащей в основе стандарта, проектирование и реализацию программного средства, осуществляющего шифрование и дешифрование данных в режимах простой замены, гаммирования и гаммирования с обратной связью, а также генерацию имитовставки и тестирование разработанного программного средства.

Конечным результатом данной лабораторной работы должна быть разработанная программа, осуществляющая шифрование и дешифрование данных по ГОСТ 28147-89 во всех перечисленных режимах как для текста, вводимого с клавиатуры, так и для файлов произвольного содержания, и вычисляющая имитовставку заданной длины.

2 ХОД РАБОТЫ

В ходе выполнения работы было создано программное средство на языке программирования *Rust*. Реализация алгоритма ГОСТ 28147-89 вынесена в отдельный модуль `encryption::gost28147_89`, взаимодействие с пользователем осуществляется через текстовое меню, позволяющее выбрать источник данных (текст или файл), выполняемую операцию (зашифрование, расшифрование или вычисление имитовставки), режим работы алгоритма и 256-битный ключ.

Алгоритм построен по схеме сети Фейстеля с 32 раундами преобразования над 64-битным блоком, разделённым на две 32-битные половины. Ключ имеет длину 256 бит и представлен восемью 32-битными подключками, порядок использования которых при зашифровании и расшифровании различается. Основная функция раунда выполняет сложение половины блока с подключком по модулю 2^{32} , замену по таблице S-блоков и циклический сдвиг влево на 11 бит.

Зашифрование и расшифрование реализованы как независимые операции, поэтому шифротекст, полученный в одном сеансе работы программы, может быть расшифрован в другом. Перед расшифрованием выполняется проверка шифротекста на соответствие выбранному режиму: в режиме простой замены его длина должна быть кратна размеру блока с учётом байта дополнения, а в режимах гаммирования шифротекст должен содержать восьмибайтовую синхропосылку. Некорректные входные данные и ошибки ввода-вывода выводятся в виде сообщения об ошибке и не прерывают работу программы.

Исходный код разработанного программного средства приведён в приложении А: реализация алгоритма представлена в листинге А.1, точка входа в программу – в листинге А.2.

2.1 Режимы ввода и вывода

Программа поддерживает два режима ввода-вывода, переключаемых командой `m`. В текстовом режиме открытый текст вводится с клавиатуры, а шифротекст выводится на экран в шестнадцатеричном виде и в таком же виде принимается при расшифровании. В файловом режиме данные читаются из файла и записываются в файл в виде последовательности байт, что позволяет обрабатывать данные произвольного содержания, в том числе двоичные. Меню программы и переключение режима ввода-вывода представлены на рисунке 2.1.

```

Input/output: text
1) encrypt (ECB)      4) decrypt (ECB)
2) encrypt (CTR)      5) decrypt (CTR)
3) encrypt (CFM)      6) decrypt (CFM)
7) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): m

Switched to file mode

Input/output: file
1) encrypt (ECB)      4) decrypt (ECB)
2) encrypt (CTR)      5) decrypt (CTR)
3) encrypt (CFM)      6) decrypt (CFM)
7) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 2

```

Рисунок 2.1 – Меню программы и переключение режима ввода-вывода

2.2 Режим простой замены

В режиме простой замены каждый 64-битный блок открытого текста преобразуется независимо от остальных, что не требует синхропосылки, но сохраняет одинаковые блоки шифротекста для одинаковых блоков открытого текста. Результат работы программы в данном режиме представлен на рисунке 2.2: открытому тексту из повторяющихся символов соответствует шифротекст с повторяющимися блоками.

```

Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 1

Plaintext: AAAAAAAAAAAAAAAAAAAAAA
mode      : ECB (encryption)
plaintext : 25 bytes
encrypted : 3D75C120DCE30E2F3D75C120DCE30E2F3D75C120DCE30E2FC694478853AAE0D407 (33 bytes)

Input/output: text
1) encrypt (ECB)      4) decrypt (ECB)
2) encrypt (CTR)      5) decrypt (CTR)
3) encrypt (CFM)      6) decrypt (CFM)
7) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 4

Ciphertext (hex): 3D75C120DCE30E2F3D75C120DCE30E2F3D75C120DCE30E2FC694478853AAE0D407
mode      : ECB (decryption)
encrypted : 33 bytes
decrypted : AAAAAAAAAAAAAAAAAAAAAA (25 bytes)

```

Рисунок 2.2 – Зашифрование и расшифрование в режиме простой замены

2.3 Режим гаммирования

В режиме гаммирования шифрование выполняется наложением на открытый текст гаммы, вырабатываемой из синхропосылки при помощи констант C_1 и C_2 . Синхропосылка вырабатывается случайным образом при каждом зашифровании и передаётся в начале шифротекста. Данный режим не требует дополнения сообщения до целого числа блоков и исключает совпадение блоков шифротекста. Результат работы программы в данном режиме представлен на рисунке 2.3.

```
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 2

Plaintext: AAAAAAAAAAAAAAAAAAAAAA
mode      : CTR (encryption)
plaintext : 25 bytes
encrypted : 846E9724EC7BB8A4C11CA7872DA11315D1F173C282C66EB3C52221F0E679A9CF89 (33 bytes)

Input/output: text
1) encrypt (ECB)      4) decrypt (ECB)
2) encrypt (CTR)      5) decrypt (CTR)
3) encrypt (CFM)      6) decrypt (CFM)
7) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 5

Ciphertext (hex): 846E9724EC7BB8A4C11CA7872DA11315D1F173C282C66EB3C52221F0E679A9CF89
mode      : CTR (decryption)
encrypted : 33 bytes
decrypted : AAAAAAAAAAAAAAAAAAAAAA (25 bytes)
```

Рисунок 2.3 – Зашифрование и расшифрование в режиме гаммирования

2.4 Режим гаммирования с обратной связью

В режиме гаммирования с обратной связью первый блок гаммы вырабатывается зашифрованием синхропосылки, а каждый последующий – зашифрованием предыдущего блока шифротекста. Благодаря этому одинаковым блокам открытого текста соответствуют различные блоки шифротекста, а искажение одного блока влияет на результат расшифрования последующих. Результат работы программы в данном режиме представлен на рисунке 2.4.

```

Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 3

Plaintext: AAAAAAAAAAAAAAAAAAAAAAAAAA
mode      : CFM (encryption)
plaintext : 25 bytes
encrypted : C7BB7706C7F2B3E3DF7682803D01AFA4E3CE382277909BD33D6526945109398662 (33 bytes)

Input/output: text
1) encrypt (ECB)      4) decrypt (ECB)
2) encrypt (CTR)      5) decrypt (CTR)
3) encrypt (CFM)      6) decrypt (CFM)
7) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 6

Ciphertext (hex): C7BB7706C7F2B3E3DF7682803D01AFA4E3CE382277909BD33D6526945109398662
mode      : CFM (decryption)
encrypted : 33 bytes
decrypted : AAAAAAAAAAAAAAAAAAAAAAAAAA (25 bytes)

```

Рисунок 2.4 – Зашифрование и расшифрование в режиме гаммирования с обратной связью

2.5 Генерация имитовставки

Имитовставка генерируется в режиме, использующем 16 раундов преобразования вместо 32, и предназначена для контроля целостности передаваемого сообщения. Длина имитовставки задаётся пользователем и не превышает 32 бит. Результат работы программы представлен на рисунке 2.5.

```

Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 7

Plaintext: AAAAAAAAAAAAAAAAAAAAAAAAAA
MAC length in bits [32]: 32
message   : 25 bytes
mac       : 42E0E3E6 (32 bits)

```

Рисунок 2.5 – Генерация имитовставки

2.6 Обработка файлов

В файловом режиме программа запрашивает путь к исходному файлу и путь к файлу результата. По умолчанию при зашифровании к имени исходного файла добавляется расширение `.gost`, а при расшифровании оно отбрасывается; если файл результата уже существует, программа запрашивает подтверждение перезаписи. Результат обработки файла в режиме гаммирования представлен на рисунке 2.6. Побайтовое сравнение исходного и расшифрованного файлов показало их полное совпадение.

```
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 2

Plaintext file: message.txt
Output file [message.txt.gost]:
  mode      : CTR (encryption)
  input     : message.txt (39 bytes)
  output    : message.txt.gost (47 bytes)

Input/output: file
1) encrypt (ECB)      4) decrypt (ECB)
2) encrypt (CTR)      5) decrypt (CTR)
3) encrypt (CFM)      6) decrypt (CFM)
7) MAC
Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to change key, 'q' to quit): 5

Ciphertext file: message.txt.gost
Output file [message.txt]: message.decrypted.txt
  mode      : CTR (decryption)
  input     : message.txt.gost (47 bytes)
  output    : message.decrypted.txt (39 bytes)
```

Рисунок 2.6 – Зашифрование и расшифрование файла

Таким образом все режимы работы описываемого стандартом ГОСТ 28147-89 алгоритма были изучены, реализованы и протестированны в полном объёме как для текста, вводимого с клавиатуры, так и для файлов.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы были изучены принципы построения симметричных блочных шифров на примере алгоритма ГОСТ 28147-89. Было разработано программное средство на языке *Rust*, реализующее шифрование и дешифрование данных в режимах простой замены, гаммирования и гаммирования с обратной связью, а также генерацию имитовставки заданной длины. Зашифрование и расшифрование выполняются как отдельные операции над текстом, вводимым с клавиатуры, либо над файлами произвольного содержания.

Тестирование показало, что расшифрование во всех реализованных режимах восстанавливает исходное сообщение без искажений, что подтверждает корректность реализации алгоритма. Сравнение режимов показало, что режим простой замены сохраняет структуру открытого текста и потому непригоден для шифрования сообщений, содержащих повторяющиеся блоки, тогда как режимы гаммирования лишены данного недостатка: в режиме гаммирования с обратной связью каждый блок гаммы вырабатывается зашифрованием предыдущего блока шифротекста, поэтому повторяющиеся блоки открытого текста не приводят к повторениям в шифротексте.

Все задачи лабораторной работы выполнены в полном объёме.

ПРИЛОЖЕНИЕ А

(обязательное)

Листинг программного кода

Листинг А.1 – Реализация алгоритма ГОСТ 28147-89 на *Rust*

```
use std::{fmt::Display, ops::Index, vec};

use rand::RngExt;

const BLOCK_SIZE: usize = 8;
const MAC_ITERATIONS: usize = 16;
pub const MAX_MAC_BITS: usize = 32;
const ITERATIONS: usize = 32;
const S_BLOCKS: [[u32; 16]; 8] = [
    [4, 10, 9, 2, 13, 8, 0, 14, 6, 11, 1, 12, 7, 15, 5, 3],
    [14, 11, 4, 12, 6, 13, 15, 10, 2, 3, 8, 1, 0, 7, 5, 9],
    [5, 8, 1, 13, 10, 3, 4, 2, 14, 15, 12, 7, 6, 0, 9, 11],
    [7, 13, 10, 1, 0, 8, 9, 15, 14, 4, 6, 12, 11, 2, 5, 3],
    [6, 12, 7, 1, 5, 15, 13, 8, 4, 10, 9, 14, 0, 3, 11, 2],
    [4, 11, 10, 0, 7, 2, 1, 13, 3, 6, 8, 5, 9, 12, 15, 14],
    [13, 11, 4, 1, 3, 15, 5, 9, 0, 10, 14, 7, 6, 8, 2, 12],
    [1, 15, 13, 0, 5, 7, 10, 4, 9, 2, 3, 14, 6, 11, 8, 12],
];

const FOUR_BIT_MASKS: [u32; 8] = [
    0x0000000F, 0x000000F0, 0x00000F00, 0x0000F000, 0x000F0000, 0x00F00000,
    0x0F000000, 0xF0000000,
];

const C1: u32 = 0x01010104;
const C2: u32 = 0x01010101;

#[derive(Clone, Copy, PartialEq, Eq)]
pub enum Gost28147_89Type {
    ECB,
    CTR,
    CFM,
}

impl Display for Gost28147_89Type {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        let name = match self {
            Gost28147_89Type::ECB => "ECB",
            Gost28147_89Type::CTR => "CTR",
            Gost28147_89Type::CFM => "CFM",
        };
        return write!(f, "{name}");
    }
}

pub struct Gost28147_89 {}

pub struct Gost28147_89Key(pub [u32; 8]);

impl Index<usize> for Gost28147_89Key {
    type Output = u32;

    fn index(&self, index: usize) -> &Self::Output {
        &self.0[index]
    }
}
```

```

impl Display for Gost28147_89Key {
    fn fmt(&self, f: &mut std::fmt::Formatter<'_>) -> std::fmt::Result {
        for &word in &self.0 {
            write!(f, "{:08X}", word)?;
        }

        Ok(())
    }
}

impl Gost28147_89 {
    fn get_subkey(key: &Gost28147_89Key, idx: usize, encrypt: bool) -> u32 {
        if idx > 31 {
            panic!("invalid index on K_{idx}")
        }
        if encrypt {
            if idx < 24 {
                return key[idx % 8];
            } else {
                return key[(32 - (idx + 1)) % 8];
            }
        } else {
            if idx < 8 {
                return key[idx];
            } else {
                return key[7 - idx % 8];
            }
        }
    }

    fn f(half: u32, subkey: u32) -> u32 {
        let sum = half.wrapping_add(subkey);
        let mut result: u32 = 0;
        for i in 0..8 {
            let four_bits = (sum & FOUR_BIT_MASKS[i]) >> (i * 4);
            let new_four_bits: u32 = S_BLOCKS[i][four_bits as usize] << (i *
4);

            result |= new_four_bits;
        }

        return result.rotate_left(11);
    }

    fn round(a: u32, b: u32, subkey: u32) -> (u32, u32) {
        return (b ^ Self::f(a, subkey), a);
    }

    fn encrypt_block(mut a: u32, mut b: u32, key: &Gost28147_89Key, encrypt:
bool) -> (u32, u32) {
        for iteration in 0..ITERATIONS {
            (a, b) = Self::round(a, b, Self::get_subkey(&key, iteration,
encrypt));
        }

        return (b, a);
    }

    fn mac_block(mut a: u32, mut b: u32, key: &Gost28147_89Key) -> (u32, u32)
{
        for iteration in 0..MAC_ITERATIONS {
            (a, b) = Self::round(a, b, Self::get_subkey(&key, iteration,

```

```

true));
    }

    return (a, b);
}

fn apply_algo(bytes: Vec<u8>, key: &Gost28147_89Key, encrypt: bool) ->
Vec<u8> {
    let blocks = bytes.chunks(BLOCK_SIZE);
    let mut result: Vec<u8> = vec![];

    for block in blocks {
        let a_bytes = block[0..(BLOCK_SIZE / 2)]
            .try_into()
            .expect("u32 can only be constructed from [u8;4]");
        let b_bytes = block[(BLOCK_SIZE / 2)..]
            .try_into()
            .expect("u32 can only be constructed from [u8;4]");
        let mut a = u32::from_ne_bytes(a_bytes);
        let mut b = u32::from_ne_bytes(b_bytes);

        (a, b) = Self::encrypt_block(a, b, key, encrypt);

        result.extend_from_slice(&a.to_ne_bytes());
        result.extend_from_slice(&b.to_ne_bytes());
    }

    return result;
}

pub fn encrypt_ecb(mut plain_bytes: Vec<u8>, key: &Gost28147_89Key) ->
Vec<u8> {
    let padding = {
        let pd = 8 - plain_bytes.len() % 8;
        if pd == 8 { 0 } else { pd }
    };
    for _ in 0..padding {
        plain_bytes.push(0);
    }

    let mut result = Self::apply_algo(plain_bytes, key, true);
    result.push(padding as u8);

    return result;
}

pub fn decrypt_ecb(mut encrypted_bytes: Vec<u8>, key: &Gost28147_89Key) -
> Vec<u8> {
    let padding = encrypted_bytes.pop().unwrap_or(0) as usize;

    let mut result = Self::apply_algo(encrypted_bytes, key, false);
    for _ in 0..padding {
        result.pop();
    }

    return result;
}

pub fn encrypt_ctr(plain_bytes: Vec<u8>, key: &Gost28147_89Key) ->
Vec<u8> {
    let mut result: Vec<u8> = vec![];
    let mut rng = rand::rng();
    let s_a: u32 = rng.random();
    let s_b: u32 = rng.random();

```

```

        result.extend_from_slice(&s_a.to_ne_bytes());
        result.extend_from_slice(&s_b.to_ne_bytes());

        let (mut n_1, mut n_2) = Self::encrypt_block(s_a, s_b, key, true);

        let blocks = plain_bytes.chunks(BLOCK_SIZE);
        for block in blocks {
            n_1 = n_1.wrapping_add(C1);
            n_2 = {
                let (sum, carry) = n_2.overflowing_add(C2);
                sum + carry as u32
            };

            let (g_1, g_2) = Self::encrypt_block(n_1, n_2, key, true);

            let mut gamma: Vec<u8> = vec![];
            gamma.extend_from_slice(&g_1.to_ne_bytes());
            gamma.extend_from_slice(&g_2.to_ne_bytes());

            for (idx, byte) in block.iter().enumerate() {
                result.push(byte ^ gamma[idx]);
            }
        }

        return result;
    }

    pub fn decrypt_ctr(encrypted_bytes: Vec<u8>, key: &Gost28147_89Key) ->
    Vec<u8> {
        let mut result: Vec<u8> = vec![];

        let s_a = u32::from_ne_bytes(
            encrypted_bytes[..4]
                .try_into()
                .expect("u32 can only be constructed from [u8;4]"),
        );
        let s_b = u32::from_ne_bytes(
            encrypted_bytes[4..8]
                .try_into()
                .expect("u32 can only be constructed from [u8;4]"),
        );

        let (mut n_1, mut n_2) = Self::encrypt_block(s_a, s_b, key, true);

        let blocks = encrypted_bytes[8..].chunks(BLOCK_SIZE);
        for block in blocks {
            n_1 = n_1.wrapping_add(C1);
            n_2 = {
                let (sum, carry) = n_2.overflowing_add(C2);
                sum + carry as u32
            };

            let (g_1, g_2) = Self::encrypt_block(n_1, n_2, key, true);

            let mut gamma: Vec<u8> = vec![];
            gamma.extend_from_slice(&g_1.to_ne_bytes());
            gamma.extend_from_slice(&g_2.to_ne_bytes());

            for (idx, byte) in block.iter().enumerate() {
                result.push(byte ^ gamma[idx]);
            }
        }
    }

```

```

        return result;
    }

    fn split_block(block: &[u8]) -> (u32, u32) {
        let a_bytes = block[..(BLOCK_SIZE / 2)]
            .try_into()
            .expect("u32 can only be constructed from [u8;4]");
        let b_bytes = block[(BLOCK_SIZE / 2)..BLOCK_SIZE]
            .try_into()
            .expect("u32 can only be constructed from [u8;4]");

        return (u32::from_ne_bytes(a_bytes), u32::from_ne_bytes(b_bytes));
    }

    pub fn encrypt_cfm(plain_bytes: Vec<u8>, key: &Gost28147_89Key) ->
    Vec<u8> {
        let mut result: Vec<u8> = vec![];
        let mut rng = rand::rng();
        let s_a: u32 = rng.random();
        let s_b: u32 = rng.random();

        result.extend_from_slice(&s_a.to_ne_bytes());
        result.extend_from_slice(&s_b.to_ne_bytes());

        let (mut n_1, mut n_2) = (s_a, s_b);

        let blocks = plain_bytes.chunks(BLOCK_SIZE);
        for block in blocks {
            let (g_1, g_2) = Self::encrypt_block(n_1, n_2, key, true);

            let mut gamma: Vec<u8> = vec![];
            gamma.extend_from_slice(&g_1.to_ne_bytes());
            gamma.extend_from_slice(&g_2.to_ne_bytes());

            let mut encrypted_block: Vec<u8> =
            Vec::with_capacity(block.len());
            for (idx, byte) in block.iter().enumerate() {
                encrypted_block.push(byte ^ gamma[idx]);
            }

            if encrypted_block.len() == BLOCK_SIZE {
                (n_1, n_2) = Self::split_block(&encrypted_block);
            }

            result.extend_from_slice(&encrypted_block);
        }

        return result;
    }

    pub fn decrypt_cfm(encrypted_bytes: Vec<u8>, key: &Gost28147_89Key) ->
    Vec<u8> {
        let mut result: Vec<u8> = vec![];

        let (mut n_1, mut n_2) =
        Self::split_block(&encrypted_bytes[..BLOCK_SIZE]);

        let blocks = encrypted_bytes[BLOCK_SIZE..].chunks(BLOCK_SIZE);
        for block in blocks {
            let (g_1, g_2) = Self::encrypt_block(n_1, n_2, key, true);

            let mut gamma: Vec<u8> = vec![];

```

```

        gamma.extend_from_slice(&g_1.to_ne_bytes());
        gamma.extend_from_slice(&g_2.to_ne_bytes());

        for (idx, byte) in block.iter().enumerate() {
            result.push(byte ^ gamma[idx]);
        }

        if block.len() == BLOCK_SIZE {
            (n_1, n_2) = Self::split_block(block);
        }
    }

    return result;
}

pub fn compute_mac(
    mut plain_bytes: Vec<u8>,
    key: &Gost28147_89Key,
    mac_bits: usize,
) -> Option<u32> {
    if mac_bits == 0 || mac_bits > MAX_MAC_BITS {
        panic!("mac bits must be in 1..={MAX_MAC_BITS}, got {mac_bits}")
    }
    if plain_bytes.is_empty() {
        return None;
    }

    let padding = {
        let pd = 8 - plain_bytes.len() % 8;
        if pd == 8 { 0 } else { pd }
    };
    for _ in 0..padding {
        plain_bytes.push(0);
    }
    let (mut s_1, mut s_2) = (0u32, 0u32);

    let blocks = plain_bytes.chunks(BLOCK_SIZE);
    for block in blocks {
        let (n_1, n_2) = (
            u32::from_ne_bytes(
                block[..(BLOCK_SIZE / 2)]
                    .try_into()
                    .expect("failed to compute mac"),
            ),
            u32::from_ne_bytes(
                block[(BLOCK_SIZE / 2)..]
                    .try_into()
                    .expect("failed to compute mac"),
            ),
        );

        (s_1, s_2) = Self::mac_block(n_1 ^ s_1, n_2 ^ s_2, key);
    }

    let mac_mask: u32 = u32::MAX >> (32 - mac_bits);

    return Some(s_2 & mac_mask);
}
}

```

Листинг А.2 – Точка входа в программу

```

mod encryption;
use encryption::gost28147_89::*;
use std::fs;
use std::io::{self, Write};
use std::path::{Path, PathBuf};

const KEY: Gost28147_89Key = Gost28147_89Key([1, 2, 3, 4, 5, 6, 7, 8]);
const DEFAULT_MAC_BITS: usize = 32;
const ENCRYPTED_EXTENSION: &str = "gost";
type CipherFn = fn(Vec<u8>, &Gost28147_89Key) -> Vec<u8>;

#[derive(Clone, Copy, PartialEq, Eq)]
enum IoMode {
    Text,
    File,
}

impl IoMode {
    fn name(&self) -> &'static str {
        match self {
            IoMode::Text => "text",
            IoMode::File => "file",
        }
    }
}

fn prompt(label: &str) -> String {
    print!("{label}");
    io::stdout().flush().expect("failed to flush stdout");

    let mut line = String::new();
    io::stdin()
        .read_line(&mut line)
        .expect("failed to read stdin");

    return line.trim_end_matches(['\r', '\n']).to_string();
}

fn to_hex(bytes: &[u8]) -> String {
    let mut hex = String::with_capacity(bytes.len() * 2);
    for byte in bytes {
        hex.push_str(&format!("{byte:02X}"));
    }

    return hex;
}

fn from_hex(text: &str) -> Result<Vec<u8>, String> {
    let digits: String = text.chars().filter(|c| !
c.is_whitespace()).collect();
    if digits.len() % 2 != 0 {
        return Err("hex input must contain an even number of
digits".to_string());
    }
    if !digits.chars().all(|c| c.is_ascii_hexdigit()) {
        return Err("hex input contains a non-hex character".to_string());
    }

    let mut bytes = Vec::with_capacity(digits.len() / 2);
    for i in (0..digits.len()).step_by(2) {
        let byte = u8::from_str_radix(&digits[i..i + 2], 16).expect("checked
to be valid hex");
        bytes.push(byte);
    }
}

```



```

    }

    return Ok(bytes);
}

fn cipher_fns(encryption_type: Gost28147_89Type) -> (CipherFn, CipherFn) {
    return match encryption_type {
        Gost28147_89Type::ECB => (Gost28147_89::encrypt_ecb,
Gost28147_89::decrypt_ecb),
        Gost28147_89Type::CTR => (Gost28147_89::encrypt_ctr,
Gost28147_89::decrypt_ctr),
        Gost28147_89Type::CFM => (Gost28147_89::encrypt_cfm,
Gost28147_89::decrypt_cfm),
    };
}

fn check_ciphertext(bytes: &[u8], encryption_type: Gost28147_89Type) ->
Result<(), String> {
    match encryption_type {
        Gost28147_89Type::ECB => {
            if bytes.is_empty() || (bytes.len() - 1) % 8 != 0 {
                return Err(format!(
                    "ECB ciphertext must be a whole number of 8-byte blocks
plus one padding byte, got {} bytes",
                    bytes.len()
                ));
            }
            let padding = bytes[bytes.len() - 1] as usize;
            if padding > 7 || (padding > 0 && bytes.len() == 1) {
                return Err(format!("invalid padding byte {padding} in ECB
ciphertext"));
            }
        }
        Gost28147_89Type::CTR | Gost28147_89Type::CFM => {
            if bytes.len() < 8 {
                return Err(format!(
                    "{encryption_type} ciphertext must start with an 8-byte
sync value, got {} bytes",
                    bytes.len()
                ));
            }
        }
    }

    return Ok(());
}

fn read_file(path: &Path) -> Result<Vec<u8>, String> {
    return fs::read(path).map_err(|err| format!("cannot read {}: {err}",
path.display()));
}

fn write_file(path: &Path, bytes: &[u8]) -> Result<(), String> {
    return fs::write(path, bytes).map_err(|err| format!("cannot write {}:
{err}", path.display()));
}

fn default_output_path(source: &Path, encrypt: bool) -> PathBuf {
    if encrypt {
        let mut name = source.as_os_str().to_os_string();
        name.push(format!(".{ENCRYPTED_EXTENSION}"));
    }

    return PathBuf::from(name);
}

```

```

    }

    if source
        .extension()
        .is_some_and(|ext| ext == ENCRYPTED_EXTENSION)
    {
        return source.with_extension("");
    }

    let mut name = source.as_os_str().to_os_string();
    name.push(".decrypted");

    return PathBuf::from(name);
}

fn ask_input_path(label: &str) -> Result<PathBuf, String> {
    let path = prompt(label);
    if path.is_empty() {
        return Err("no input file given".to_string());
    }

    return Ok(PathBuf::from(path));
}

fn ask_output_path(default: &Path) -> Result<PathBuf, String> {
    let answer = prompt(&format!("Output file [{}]: ", default.display()));
    let path = if answer.is_empty() {
        default.to_path_buf()
    } else {
        PathBuf::from(answer)
    };

    if path.exists() {
        let confirmation = prompt(&format!(
            "{} already exists, overwrite? [y/N]: ",
            path.display()
        ));
        if !confirmation.eq_ignore_ascii_case("y") {
            return Err("cancelled, the output file was left
untouched".to_string());
        }
    }

    return Ok(path);
}

fn read_payload(io_mode: IoMode, encrypt: bool) -> Result<(Vec<u8>,
Option<PathBuf>), String> {
    match io_mode {
        IoMode::Text => {
            let bytes = if encrypt {
                prompt("Plaintext: ").into_bytes()
            } else {
                from_hex(&prompt("Ciphertext (hex): "))?
            };

            return Ok((bytes, None));
        }
        IoMode::File => {
            let path = ask_input_path(if encrypt {
                "Plaintext file: "
            } else {
                "Ciphertext file: "
            });

```

```

        })?;
        let bytes = read_file(&path)?;

        return Ok((bytes, Some(path)));
    }
}

fn run_cipher(
    io_mode: IoMode,
    encrypt: bool,
    encryption_type: Gost28147_89Type,
    key: &Gost28147_89Key,
) -> Result<(), String> {
    let (encrypt_fn, decrypt_fn) = cipher_fns(encryption_type);
    let cipher_fn = if encrypt { encrypt_fn } else { decrypt_fn };

    let (input, source) = read_payload(io_mode, encrypt)?;
    if input.is_empty() {
        return Err("nothing to process: the input is empty".to_string());
    }
    if !encrypt {
        check_ciphertext(&input, encryption_type)?;
    }

    let destination = match &source {
        Some(path) => Some(ask_output_path(&default_output_path(path,
encrypt)?)),
        None => None,
    };

    let output = cipher_fn(input.clone(), key);

    println!(
        "    mode      : {encryption_type} ({})",
        if encrypt { "encryption" } else { "decryption" }
    );

    match source {
        Some(path) => {
            let destination = destination.expect("a file source always has a
destination");
            write_file(&destination, &output)?;

            println!("    input      : {} ({} bytes)", path.display(),
input.len());
            println!(
                "    output     : {} ({} bytes)",
                destination.display(),
                output.len()
            );
        }
        None => {
            if encrypt {
                println!("    plaintext : {} bytes", input.len());
                println!("    encrypted : {} ({} bytes)", to_hex(&output),
output.len());
            } else {
                println!("    encrypted : {} bytes", input.len());
                match String::from_utf8(output.clone()) {
                    Ok(text) => println!("    decrypted : {text} ({} bytes)",
output.len()),
                    Err(_) => println!(

```

```

        "  decrypted : <not valid UTF-8> {} ({} bytes)",
        to_hex(&output),
        output.len()
    ),
    },
    }
}

return Ok(());
}

fn run_mac(io_mode: IoMode, key: &Gost28147_89Key) -> Result<(), String> {
    let (message, source) = read_payload(io_mode, true)?;

    let bits_input = prompt(&format!("MAC length in bits
[DEFAULT_MAC_BITS]: "));
    let mac_bits = if bits_input.is_empty() {
        DEFAULT_MAC_BITS
    } else {
        match bits_input.parse::<usize>() {
            Ok(bits) if (1..=MAX_MAC_BITS).contains(&bits) => bits,
            _ => {
                return Err(format!(
                    "MAC length must be an integer in 1..={MAX_MAC_BITS}"
                ));
            }
        }
    };

    let mac = Gost28147_89::compute_mac(message.clone(), key, mac_bits)
        .ok_or_else(|| "an empty message has no MAC".to_string())?;

    match source {
        Some(path) => println!("  message : {} ({} bytes)", path.display(),
message.len()),
        None => println!("  message : {} bytes", message.len()),
    }
    println!("  mac      : {mac:08X} ({} bits)", mac_bits);

    return Ok(());
}

fn parse_key(hex_key: &str) -> Result<Gost28147_89Key, String> {
    if hex_key.len() != 64 {
        return Err("valid key should be 256 bits (64
characters)".to_string());
    }

    let bytes = from_hex(hex_key)?;
    let mut new_key = [0u32; 8];
    for (idx, word) in bytes.chunks(4).enumerate() {
        new_key[idx] = u32::from_be_bytes(word.try_into().expect("64 hex
digits give 8 words"));
    }

    return Ok(Gost28147_89Key(new_key));
}

fn main() {
    let mut key = KEY;
    let mut io_mode = IoMode::Text;

```

```

loop {
  println!();
  println!("Input/output: {}", io_mode.name());
  println!("1) encrypt (ECB)      4) decrypt (ECB)");
  println!("2) encrypt (CTR)      5) decrypt (CTR)");
  println!("3) encrypt (CFM)      6) decrypt (CFM)");
  println!("7) MAC");

  let choice = prompt(
    "Choice (or 'm' to switch text/file mode, 'k' to see key, 'c' to
change key, 'q' to quit): ",
  );
  println!();

  let result = match choice.as_str() {
    "1" => run_cipher(io_mode, true, Gost28147_89Type::ECB, &key),
    "2" => run_cipher(io_mode, true, Gost28147_89Type::CTR, &key),
    "3" => run_cipher(io_mode, true, Gost28147_89Type::CFM, &key),
    "4" => run_cipher(io_mode, false, Gost28147_89Type::ECB, &key),
    "5" => run_cipher(io_mode, false, Gost28147_89Type::CTR, &key),
    "6" => run_cipher(io_mode, false, Gost28147_89Type::CFM, &key),
    "7" => run_mac(io_mode, &key),
    "m" => {
      io_mode = match io_mode {
        IoMode::Text => IoMode::File,
        IoMode::File => IoMode::Text,
      };
      println!("Switched to {} mode", io_mode.name());
      Ok(())
    },
    "k" => {
      println!("Key is: {key}");
      Ok(())
    },
    "c" => match parse_key(prompt("Key: ").as_str()) {
      Ok(new_key) => {
        key = new_key;
        Ok(())
      },
      Err(err) => Err(err),
    },
    "q" => break,
    _ => Err(format!("unknown choice: {choice:?}")),
  };

  if let Err(err) = result {
    println!("Error: {err}");
  }
}
}

```